

ASSIGNMENT NO. 04

Q1.

Code: **#include** <iostream>

#include <string>

#include <stdexcept>

using namespace std;

class LedgerEntry {

private:

string description;

double* amounts;

int days;

static int *totalEntries*;

public:

LedgerEntry(**const string**& desc, **int** d) : description(desc), days(d) {

 amounts = **new double**[days];

for (**int** i = 0; i < days; i++) {

 amounts[i] = 0.0;

 }

totalEntries++;

 cout << "[LedgerEntry Created] " << description << " (" << days << " days)" << **endl**;

 }

LedgerEntry(**const LedgerEntry**& other) : description(other.description),
days(other.days) {

 amounts = **new double**[days];

for (**int** i = 0; i < days; i++) {

 amounts[i] = other.amounts[i];

 }

totalEntries++;

```

    cout << "[Copy Constructor] Deep copied: " << description << endl;
}

LedgerEntry(LedgerEntry&& other) noexcept
: description(other.description), amounts(other.amounts), days(other.days) {
    cout << "[Move Constructor] Ownership transferred from: " << description << endl;
    other.amounts = nullptr;
    other.days = 0;
    totalEntries++;
}

```

```

LedgerEntry& operator=(const LedgerEntry& other) {
    if (this != &other) {
        delete[] amounts;
        description = other.description;
        days = other.days;
        amounts = new double[days];
        for (int i = 0; i < days; i++) {
            amounts[i] = other.amounts[i];
        }
        cout << "[Copy Assignment] Deep copied: " << description << endl;
    }
    return *this;
}

```

```

LedgerEntry& operator=(LedgerEntry&& other) noexcept {
    if (this != &other) {
        delete[] amounts;
        description = other.description;
        amounts = other.amounts;
        days = other.days;
    }
}

```

```

        cout << "[Move Assignment] Ownership transferred from: " << description << endl;

        other.amounts = nullptr;

        other.days = 0;
    }

    return *this;
}

~LedgerEntry() {
    if (amounts != nullptr) {
        cout << "[Destructor] " << description << " destroyed" << endl;
    }

    delete[] amounts;

    totalEntries--;
}

bool isEmpty() const {
    return amounts == nullptr;
}

double sumTotal() const {
    double total = 0.0;

    for (int i = 0; i < days; i++) total += amounts[i];

    return total;
}

LedgerEntry operator+(const LedgerEntry& other) const {
    int n = (days < other.days) ? days : other.days;

    LedgerEntry result("Combined", n);

    for (int i = 0; i < n; i++) {
        result.amounts[i] = amounts[i] + other.amounts[i];
    }

    return result;
}

```

```

}

bool operator==(const LedgerEntry& other) const {
    return sumTotal() == other.sumTotal();
}

bool operator>(const LedgerEntry& other) const {
    return sumTotal() > other.sumTotal();
}

double& operator[](int index) {
    if (index < 0 || index >= days) {
        throw out_of_range("Index out of range in LedgerEntry");
    }
    return amounts[index];
}

const double& operator[](int index) const {
    if (index < 0 || index >= days) {
        throw out_of_range("Index out of range in LedgerEntry");
    }
    return amounts[index];
}

static int getTotalEntries() {
    return totalEntries;
}

friend ostream& operator<<(ostream& out, const LedgerEntry& entry);

friend istream& operator>>(istream& in, LedgerEntry& entry);
};

int LedgerEntry::totalEntries = 0;

ostream& operator<<(ostream& out, const LedgerEntry& entry) {
    out << entry.description << " : [";

```

```

    for (int i = 0; i < entry.days; i++) {
        out << entry.amounts[i];

        if (i != entry.days - 1) out << ", ";
    }

    out << "]" Total: " << entry.sumTotal();

    return out;
}

istream& operator>>(istream& in, LedgerEntry& entry) {
    for (int i = 0; i < entry.days; i++) {
        in >> entry.amounts[i];
    }

    return in;
}

int main() {

    LedgerEntry jan("January Sales", 5);
    jan[0] = 1200.50; jan[1] = 3400.00; jan[2] = 800.75;
    jan[3] = 2100.00; jan[4] = 650.25;

    LedgerEntry feb("February Sales", 5);
    feb[0] = 900.00; feb[1] = 2200.50; feb[2] = 1750.00;
    feb[3] = 3000.00; feb[4] = 475.50;

    cout << jan << endl;

    cout << feb << endl;

    LedgerEntry combined = jan + feb;

    cout << "Combined: " << combined << endl;

    cout << "Jan == Feb : " << (jan == feb ? "Yes" : "No") << endl;

    cout << "Jan > Feb : " << (jan > feb ? "Yes" : "No") << endl;

    LedgerEntry moved = move(jan);

```

```

    cout << "After move, jan.amounts is null: " << (jan.isEmpty() ? "YES" : "NO") << endl;

    cout << "Moved entry: " << moved << endl;

    LedgerEntry q1("Q1 Total", 5);

    q1 = move(feb);

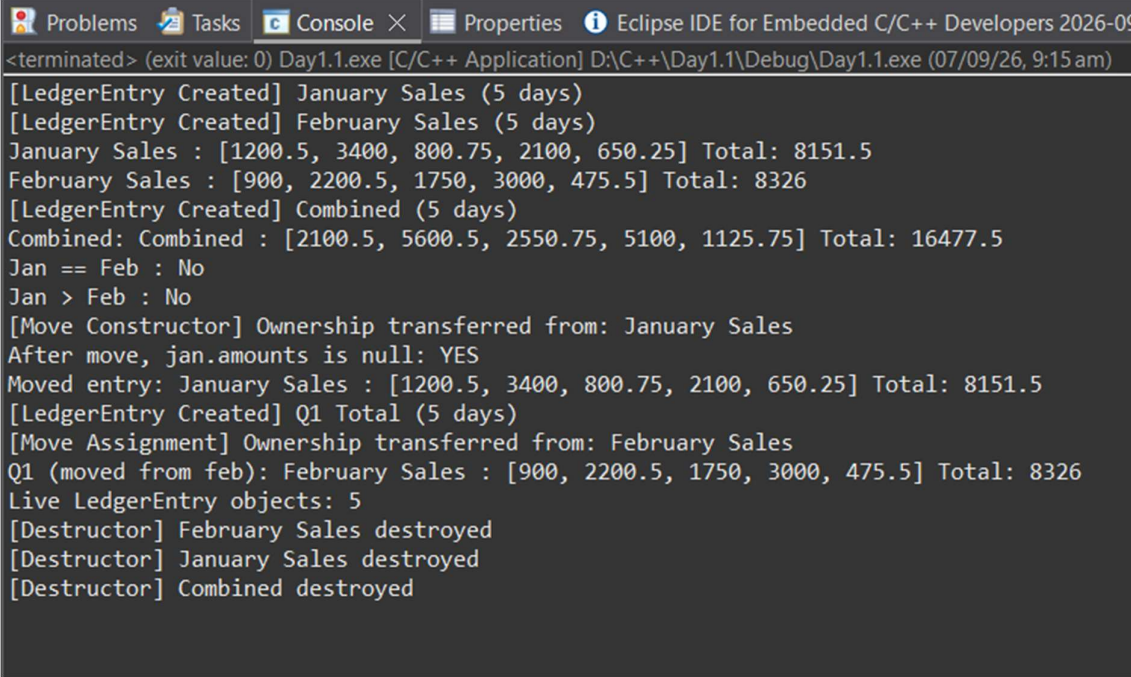
    cout << "Q1 (moved from feb): " << q1 << endl;

    cout << "Live LedgerEntry objects: " << LedgerEntry::getTotalEntries() << endl;

    return 0;
}

```

Output:



```

<terminated> (exit value: 0) Day1.1.exe [C/C++ Application] D:\C++\Day1.1\Debug\Day1.1.exe (07/09/26, 9:15 am)
[LedgerEntry Created] January Sales (5 days)
[LedgerEntry Created] February Sales (5 days)
January Sales : [1200.5, 3400, 800.75, 2100, 650.25] Total: 8151.5
February Sales : [900, 2200.5, 1750, 3000, 475.5] Total: 8326
[LedgerEntry Created] Combined (5 days)
Combined: Combined : [2100.5, 5600.5, 2550.75, 5100, 1125.75] Total: 16477.5
Jan == Feb : No
Jan > Feb : No
[Move Constructor] Ownership transferred from: January Sales
After move, jan.amounts is null: YES
Moved entry: January Sales : [1200.5, 3400, 800.75, 2100, 650.25] Total: 8151.5
[LedgerEntry Created] Q1 Total (5 days)
[Move Assignment] Ownership transferred from: February Sales
Q1 (moved from feb): February Sales : [900, 2200.5, 1750, 3000, 475.5] Total: 8326
Live LedgerEntry objects: 5
[Destructor] February Sales destroyed
[Destructor] January Sales destroyed
[Destructor] Combined destroyed

```

Q2.

Code: /*

* 4.2.cpp

*

* Created on: 07-Sept-2026

* Author: Sakshi

```

*/

#include <iostream>

#include <string>

#include <vector>

#include <iomanip>

using namespace std;

class Vehicle {

protected:

    string registrationNo;

    string ownerName;

    int yearOfManufacture;

    double kmDriven;

public:

    Vehicle(const string& reg, const string& owner, int year, double km)

        : registrationNo(reg), ownerName(owner), yearOfManufacture(year), kmDriven(km) {

        cout << "[Vehicle Constructor] " << registrationNo << " : " << ownerName << endl;

    }

    virtual double fuelCost(double kmToTravel) const = 0;

    virtual string vehicleType() const = 0;

    virtual void describe() const {

        cout << "Reg No: " << registrationNo << ", Owner: " << ownerName

            << ", Year: " << yearOfManufacture << ", Km Driven: " << kmDriven;

    }

    virtual ~Vehicle() {

        cout << "[Vehicle Destructor] " << registrationNo << endl;

    }

    double getKmDriven() const { return kmDriven; }

    string getRegNo() const { return registrationNo; }

```

```

    string getOwnerName() const { return ownerName; }

};

class Car : public Vehicle {
private:
    string fuelType;
    double mileageKmpl;
public:
    Car(const string& reg, const string& owner, int year, double km,
        const string& fuel, double mileage)
        : Vehicle(reg, owner, year, km), fuelType(fuel), mileageKmpl(mileage) {
        cout << "[Car Constructor] " << reg << endl;
    }

    double fuelCost(double kmToTravel) const override {
        double pricePerLitre = (fuelType == "Petrol") ? 106.00 : 93.00;
        return (kmToTravel / mileageKmpl) * pricePerLitre;
    }

    string vehicleType() const override { return "Car"; }

    void describe() const override {
        Vehicle::describe();

        cout << ", Fuel: " << fuelType << ", Mileage: " << mileageKmpl << " kmpl";
    }

    ~Car() override {
        cout << "[Car Destructor] " << registrationNo << endl;
    }
};

class Truck : public Vehicle {
protected:
    double payloadCapacityTons;

```



```

    double fuelEfficiencyKmpl;

public:
    Truck(const string& reg, const string& owner, int year, double km,
          double payload, double efficiency)
        : Vehicle(reg, owner, year, km), payloadCapacityTons(payload),
        fuelEfficiencyKmpl(efficiency) {
        cout << "[Truck Constructor] " << reg << endl;
    }

    double fuelCost(double kmToTravel) const override {
        double adjustedEfficiency = fuelEfficiencyKmpl * (1.0 - 0.05 *
        payloadCapacityTons);
        if (adjustedEfficiency <= 0) adjustedEfficiency = 1.0;
        double pricePerLitre = 93.00;
        return (kmToTravel / adjustedEfficiency) * pricePerLitre;
    }

    string vehicleType() const override { return "Truck"; }

    void describe() const override {
        Vehicle::describe();
        cout << ", Payload: " << payloadCapacityTons << " tons";
    }

    ~Truck() override {
        cout << "[Truck Destructor] " << registrationNo << endl;
    }
};

class ElectricTruck : public Truck {
private:
    double batteryCapacityKWh;
    double rangePerChargeKm;
public:

```

```

ElectricTruck(const string& reg, const string& owner, int year, double km,
    double payload, double efficiency,
    double battery, double range)
: Truck(reg, owner, year, km, payload, efficiency),
  batteryCapacityKWh(battery), rangePerChargeKm(range) {
    cout << "[ElectricTruck Constructor] " << reg << endl;
}

double fuelCost(double kmToTravel) const override {
    double electricityCostPerUnit = 9.50;

    return (kmToTravel / rangePerChargeKm) * batteryCapacityKWh *
electricityCostPerUnit;
}

string vehicleType() const override { return "Electric Truck"; }

void describe() const override {
    Truck::describe();

    cout << ", Battery: " << batteryCapacityKWh << " kWh, Range: " <<
rangePerChargeKm << " km";
}

double getBatteryPercent() const {
    return 85.0;
}

~ElectricTruck() override {
    cout << "[ElectricTruck Destructor] " << registrationNo << endl;
}

};

class Van : public Vehicle {
private:
    int seatingCapacity;
    double mileageKmpl;

```

public:

```
Van(const string& reg, const string& owner, int year, double km,
    int seats, double mileage)
    : Vehicle(reg, owner, year, km), seatingCapacity(seats), mileageKmpl(mileage) {
    cout << "[Van Constructor] " << reg << endl;
}

double fuelCost(double kmToTravel) const override {
    double pricePerLitre = 106.00; // standard petrol calculation
    return (kmToTravel / mileageKmpl) * pricePerLitre;
}

string vehicleType() const override { return "Van"; }

void describe() const override {
    Vehicle::describe();
    cout << ", Seats: " << seatingCapacity;
}

~Van() override {
    cout << "[Van Destructor] " << registrationNo << endl;
}

};

void printFleetReport(const vector<Vehicle*>& fleet, double tripKm) {
    cout << "\n===== FLEET REPORT - Trip Distance: " << tripKm << " km =====" << endl;
    cout << left << setw(12) << "Reg" << setw(18) << "Type"
        << setw(18) << "Owner" << "Km Driven" << endl;
    for (auto* v : fleet) {
        cout << left << setw(12) << v->getRegNo() << setw(18) << v->vehicleType()
            << setw(18) << v->getOwnerName() << v->getKmDriven() << endl;
    }
    cout << "\n===== FUEL / CHARGE COST ESTIMATE =====" << endl;
```

```

Vehicle* cheapest = nullptr;

double lowestCost = 0.0;

for (auto* v : fleet) {

    double cost = v->fuelCost(tripKm);

    cout << v->getRegNo() << " (" << v->vehicleType() << ") : Rs. "

        << fixed << setprecision(2) << cost;

    ElectricTruck* et = dynamic_cast<ElectricTruck*>(v);

    if (et != nullptr) {

        cout << "[Battery: " << et->getBatteryPercent() << "% charged]";

    }

    cout << endl;

    if (cheapest == nullptr || cost < lowestCost) {

        cheapest = v;

        lowestCost = cost;

    }

}

if (cheapest != nullptr) {

    cout << "\nMost Efficient Vehicle: " << cheapest->getRegNo()

        << " (" << cheapest->vehicleType() << ") - Rs. "

        << fixed << setprecision(2) << lowestCost << " for " << tripKm << " km" << endl;

}

}

int main() {

    cout << "----- Part B: Constructor/Destructor order for one ElectricTruck -----" << endl;

    {

        ElectricTruck et("V-ET001", "Green Logistics", 2023, 5000, 8.0, 6.0, 150.0, 200.0);

    }

    cout << "\n----- Part C: Polymorphic Fleet Report -----" << endl;

```

```
vector<Vehicle*> fleet;

fleet.push_back(new Car("KA01AA001", "Ramesh Kumar", 2020, 45200, "Petrol",
18.0));

fleet.push_back(new Truck("MH04BB002", "Shyam Logistics", 2019, 123500, 5.0,
6.5));

fleet.push_back(new ElectricTruck("GJ07CC003", "Green Fleet Co", 2022, 89000, 4.0,
7.0, 150.0, 200.0));

fleet.push_back(new Van("RJ14DD004", "Suresh Transport", 2021, 30500, 12, 15.0));

fleet.push_back(new Car("DL08EE005", "Priya Sharma", 2022, 22000, "Diesel", 22.0));

printFleetReport(fleet, 200);

for (auto* v : fleet) {

    delete v;

}

fleet.clear();

return 0;

}
```

Output:

```
----- Part B: Constructor/Destructor order for one ElectricTruck -----
[Vehicle Constructor] V-ET001 : Green Logistics
[Truck Constructor] V-ET001
[ElectricTruck Constructor] V-ET001
[ElectricTruck Destructor] V-ET001
[Truck Destructor] V-ET001
[Vehicle Destructor] V-ET001

----- Part C: Polymorphic Fleet Report -----
[Vehicle Constructor] KA01AA001 : Ramesh Kumar
[Car Constructor] KA01AA001
[Vehicle Constructor] MH04BB002 : Shyam Logistics
[Truck Constructor] MH04BB002
[Vehicle Constructor] GJ07CC003 : Green Fleet Co
[Truck Constructor] GJ07CC003
[ElectricTruck Constructor] GJ07CC003
[Vehicle Constructor] RJ14DD004 : Suresh Transport
[Van Constructor] RJ14DD004
[Vehicle Constructor] DL08EE005 : Priya Sharma
[Car Constructor] DL08EE005
|
===== FLEET REPORT - Trip Distance: 200 km =====
Reg           Type           Owner           Km Driven
KA01AA001     Car           Ramesh Kumar     45200
MH04BB002     Truck        Shyam Logistics  123500
GJ07CC003     Electric Truck Green Fleet Co   89000
RJ14DD004     Van          Suresh Transport 30500
DL08EE005     Car          Priya Sharma     22000

===== FUEL / CHARGE COST ESTIMATE =====
KA01AA001 (Car) : Rs. 1177.78
MH04BB002 (Truck) : Rs. 3815.38
GJ07CC003 (Electric Truck) : Rs. 1425.00 [Battery: 85.00% charged]
RJ14DD004 (Van) : Rs. 1413.33
DL08EE005 (Car) : Rs. 845.45

Most Efficient Vehicle: DL08EE005 (Car) - Rs. 845.45 for 200.00 km
[Car Destructor] KA01AA001
[Vehicle Destructor] KA01AA001
[Truck Destructor] MH04BB002
[Vehicle Destructor] MH04BB002
[ElectricTruck Destructor] GJ07CC003
[Truck Destructor] GJ07CC003
[Vehicle Destructor] GJ07CC003
```

Q3

Code:

```
#include <iostream>

#include <string>

#include <vector>

#include <stdexcept>

#include <cctype>

using namespace std;

class DataProcessor {

public:

    virtual void loadData(const string& source) = 0;

    virtual void processData() = 0;

    virtual void exportResult(const string& destination) = 0;

    virtual string processorType() const = 0;

    virtual int recordCount() const = 0;

    virtual void printSummary() const {

        cout << "[Summary] Processor: " << processorType()

            << " | Records: " << recordCount() << endl;

    }

    virtual ~DataProcessor() = default;

};

class CSVProcessor : public DataProcessor {

private:

    vector<string> records;

public:

    void loadData(const string& source) override {

        records.clear();

        records.push_back("john,25,engineer");
```

```

        records.push_back("mary,30,doctor");
        records.push_back("alex,28,teacher");
        records.push_back("sara,35,artist");
        records.push_back("mike,40,lawyer");

        cout << "Loaded " << records.size() << " rows from " << source << endl;
    }

    void processData() override {
        for (auto& row : records) {
            for (auto& ch : row) {
                ch = toupper(ch);
            }
        }
    }

    void exportResult(const string& destination) override {
        cout << "[CSV EXPORT -> " << destination << "]" << endl;

        for (auto& row : records) {
            cout << row << endl;
        }
    }

    string processorType() const override { return "CSV Processor"; }

    int recordCount() const override { return (int)records.size(); }
};

class SensorStreamProcessor : public DataProcessor {
private:
    vector<double> readings;

    double meanVal, minVal, maxVal;

public:
    SensorStreamProcessor() : meanVal(0), minVal(0), maxVal(0) {}

```



```

void loadData(const string& source) override {
    readings = {23.4, 21.8, 25.1, 22.6, 24.0, 23.9, 21.5, 26.2};
    cout << "Loaded " << readings.size() << " sensor readings from " << source << endl;
}

void processData() override {
    double sum = 0;
    minVal = readings[0];
    maxVal = readings[0];
    for (double r : readings) {
        sum += r;
        if (r < minVal) minVal = r;
        if (r > maxVal) maxVal = r;
    }
    meanVal = sum / readings.size();
}

void exportResult(const string& destination) override {
    cout << "[Sensor Export -> " << destination << "]" << endl;
    cout << "Mean: " << meanVal << ", Min: " << minVal << ", Max: " << maxVal << endl;
}

string processorType() const override { return "Sensor Stream Processor"; }

int recordCount() const override { return (int)readings.size(); }

    double getMean() const { return meanVal; }
};

template <typename T>
class DataBuffer {

```

private:

T* data;

int capacity;

int head;

int tail;

int count;

public:

DataBuffer(int cap) : capacity(cap), head(0), tail(0), count(0) {

 data = **new T**[capacity];

}

~DataBuffer() {

delete[] data;

}

void push(const T& value) {

 data[tail] = value;

 tail = (tail + 1) % capacity;

if (count == capacity) {

 head = (head + 1) % capacity;

else {

 count++;

 }

}

T pop() {

if (isEmpty()) {

throw underflow_error("Buffer is empty");

 }

T value = data[head];

 head = (head + 1) % capacity;

```

        count--;

        return value;
    }

    T peek() const {
        if (isEmpty()) {
            throw underflow_error("Buffer is empty");
        }

        return data[head];
    }

    bool isEmpty() const { return count == 0; }

    bool isFull() const { return count == capacity; }

    int size() const { return count; }

    template <typename U>
    friend ostream& operator<<(ostream& out, const DataBuffer<U>& buf);
};

template <typename U>
ostream& operator<<(ostream& out, const DataBuffer<U>& buf) {
    out << "[";

    int idx = buf.head;

    for (int i = 0; i < buf.count; i++) {
        out << buf.data[idx];

        if (i != buf.count - 1) out << ", ";

        idx = (idx + 1) % buf.capacity;
    }

    out << "]";

    return out;
}

int main() {

```

```

cout << "----- Part A: Abstract DataProcessor Pipeline -----" << endl;

vector<DataProcessor*> pipeline;

pipeline.push_back(new CSVProcessor());

pipeline.push_back(new SensorStreamProcessor());

for (auto* p : pipeline) {

    p->loadData("source_data");

    p->processData();

    p->printSummary();

    p->exportResult("output_dir");

    cout << "---" << endl;

}

for (auto* p : pipeline) {

    if (auto* csv = dynamic_cast<CSVProcessor*>(p)) {

        cout << "Found CSV Processor with " << csv->recordCount() << " records." <<
endl;

    } else if (auto* sensor = dynamic_cast<SensorStreamProcessor*>(p)) {

        cout << "Found Sensor Processor - Mean: " << sensor->getMean() << endl;

    }

}

for (auto* p : pipeline) {

    delete p;

}

pipeline.clear();

cout << "\n----- Part B: DataBuffer<T> Demo -----" << endl;

DataBuffer<int> tickBuffer(5);

for (int i = 1; i <= 7; i++) tickBuffer.push(i * 10);

cout << "Tick Buffer: " << tickBuffer << endl;

DataBuffer<double> tempBuffer(4);

```

```

tempBuffer.push(36.6); tempBuffer.push(37.1);

tempBuffer.push(38.2); tempBuffer.push(36.9);

cout << "Before pop: " << tempBuffer << endl;

cout << "Popped: " << tempBuffer.pop() << endl;

cout << "After pop: " << tempBuffer << endl;

DataBuffer<string> logBuffer(3);

logBuffer.push("INFO: Server started");

logBuffer.push("WARN: High memory usage");

logBuffer.push("ERROR: DB connection timeout");

logBuffer.push("INFO: Retry successful");

cout << "Log Buffer: " << logBuffer << endl;

return 0;

}

```

Output:

```

terminated - (Exit Value: 0) Debug Console [C:\C++\Application\Debug\Debug.exe (07/05/20, 3:52am)]
----- Part A: Abstract DataProcessor Pipeline -----
Loaded 5 rows from source_data
[Summary] Processor: CSV Processor | Records: 5
[CSV EXPORT -> output_dir]
JOHN,25,ENGINEER
MARY,30,DOCTOR
ALEX,28,TEACHER
SARA,35,ARTIST
MIKE,40,LAWYER
---
Loaded 8 sensor readings from source_data
[Summary] Processor: Sensor Stream Processor | Records: 8
[Sensor Export -> output_dir]
Mean: 23.5625, Min: 21.5, Max: 26.2
---
Found CSV Processor with 5 records.
Found Sensor Processor - Mean: 23.5625

----- Part B: DataBuffer<T> Demo -----
Tick Buffer: [30, 40, 50, 60, 70]
Before pop: [36.6, 37.1, 38.2, 36.9]
Popped: 36.6
After pop: [37.1, 38.2, 36.9]
Log Buffer: [WARN: High memory usage, ERROR: DB connection timeout, INFO: Retry successful]

```